

爬虫系统PPT演示大纲

第一页：项目概述

标题：智能爬虫数据分析与可视化平台

项目背景

- 互联网时代数据获取需求日益增长
- 传统爬虫工具缺乏智能化分析和可视化能力
- 多用户协作与数据管理需求

核心功能

- 可视化配置爬虫任务
- AI智能数据分析（本地/云端双模式）
- 多维度数据可视化展示
- 多用户账号与数据隔离

第二页：系统分层架构

架构总览

层级	技术组件	职责说明
表现层	HTML5 + CSS3 + JavaScript	用户交互界面，配置编辑，结果展示
路由层	Flask Routes	HTTP请求处理，URL映射，权限校验
业务逻辑层	CrawlerService / AIModelCaller / DataVisualizer	爬虫调度、AI分析、图表生成
数据访问层	Model Classes	数据库CRUD操作封装
数据存储层	SQLite + File System	结构化数据与文件存储

架构特点

- 分层清晰，职责单一
- 模块间松耦合，便于维护扩展
- 支持水平扩展（云服务化准备）

数据流向

用户请求 → Flask路由 → 业务服务 → 数据模型 → SQLite数据库
↓
返回JSON/图表Base64 ← 前端渲染展示

第三页：团队分工

团队成员与职责

成员	负责模块	具体工作内容	核心文件
A	爬虫内容解析与后端数据处理	1. 爬虫引擎开发 2. HTML内容解析 3. 数据清洗与存储 4. 任务调度管理	crawler_service.py crawler_task.py crawler_result.py
B	AI本地模型嵌入与API通信	1. 本地模型调用封装 2. API接口通信 3. 提示词工程 4. 结果解析处理	call_ai.py ai_config.py
C	Matplotlib可视化与前端美化	1. 图表生成（6种类型） 2. 中文显示优化 3. 页面UI设计 4. 交互逻辑实现	show.py .html .css *.js

协作接口定义

A与B的协作接口

```
# A调用B进行数据分析
from call_ai import analyze_crawler_data

result = analyze_crawler_data(
    crawler_data=raw_data,
    task_type="sentiment", # 分析类型
    ai_config=user_ai_config # 用户AI配置
)
```

A与C的协作接口

```
# A调用C生成可视化
from show import visualize_data

charts = visualize_data(
    data=processed_data,
    chart_types=["wordcloud", "sentiment_pie", "keyword_bar"]
)
```

B与C的协作接口

```
# B的分析结果作为C的输入
ai_result = {
    "sentiment_distribution": {...}, # B生成
    "keywords": [...],             # B生成
    "categories": {...}            # B生成
}
# C根据B的结果生成对应图表
```

第四页：爬虫JSON解析策略

配置文件结构

爬虫配置 (crawler_config.json)

```
{
  "type": "crawler",
  "name": "豆瓣图书爬虫",
  "description": "爬取豆瓣图书评论信息",

  "urls": [
    "https://book.douban.com/subject/xxxx/reviews",
    "https://book.douban.com/subject/yyyy/reviews"
  ],

  "selectors": {
    "title": "h1 span",
    "reviews": ".review-list .review-item",
    "review_content": ".short-content",
    "rating": ".main-title-rating",
    "author": ".name",
    "book_info": {
      "author": "#info a[href*='author']",
      "publisher": "#info span.pl:contains('出版社') + a",
      "price": "#info span.pl:contains('定价') + text()"
    }
  },

  "headers": {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
    "Accept": "text/html,application/xhtml+xml",
    "Accept-Language": "zh-CN,zh;q=0.9"
  },

  "delay": {
    "min": 2,
    "max": 5,
    "random": true
  },
}
```

```
"max_pages": 10,
"timeout": 30,
"retry_times": 3,
"retry_delay": 5
}
```

时间配置 (time_config.json)

```
{
  "type": "time",
  "name": "每日定时任务",
  "cron": "0 9 * * *",
  "timezone": "Asia/Shanghai",
  "description": "每天早上9点执行爬虫任务"
}
```

解析策略详解

1. 选择器解析策略

选择器类型	示例	用途
简单选择器	"title": "h1 span"	提取单一元素文本
列表选择器	"reviews": ".review-item"	提取多个相同元素
嵌套选择器	"book_info": {...}	提取结构化数据
属性选择器	"links": "a[href]"	提取元素属性

2. 嵌套选择器解析流程

```
# 伪代码展示解析逻辑
for key, selector in selectors.items():
    if isinstance(selector, dict):
        # 嵌套选择器处理
        nested_data = {}
        for sub_key, sub_selector in selector.items():
            elements = soup.select(sub_selector)
            nested_data[sub_key] = [e.get_text() for e in elements]
        result[key] = nested_data
    else:
        # 简单选择器处理
        elements = soup.select(selector)
        result[key] = [e.get_text() for e in elements]
```

3. 动态参数解析

参数	类型支持	说明
delay	int / dict	固定延时或随机延时范围
max_pages	int	最大爬取页数限制
timeout	int	请求超时时间
retry_times	int	失败重试次数
retry_delay	int	重试间隔时间

4. 配置验证策略

```
def validate_config(config):
    required_fields = ['type', 'name', 'urls', 'selectors']
    for field in required_fields:
        if field not in config:
            raise ValueError(f"缺少必填字段: {field}")

    # URLs验证
    if not isinstance(config['urls'], list) or len(config['urls']) == 0:
        raise ValueError("urls必须是非空列表")

    # 选择器验证
    if not isinstance(config['selectors'], dict):
        raise ValueError("selectors必须是字典")
```

第五页：数据存储策略

存储架构

双模式存储设计

存储模式	存储位置	生命周期	适用场景
暂存模式	内存/临时文件	任务完成后短期保留	临时查看、快速验证
长期保存	SQLite数据库	永久保存	重要数据、历史归档

多用户数据隔离

数据库设计

```
-- 用户表
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL, -- bcrypt加密
```

```

        create_time DATETIME DEFAULT CURRENT_TIMESTAMP
    );

-- 配置文件表 (用户隔离)
CREATE TABLE config_files (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(20) NOT NULL, -- 'crawler' 或 'time'
    content TEXT NOT NULL,      -- JSON配置内容
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users (id)
);

-- 爬虫任务表 (用户隔离)
CREATE TABLE crawler_tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    config_id INTEGER NOT NULL,
    time_config_id INTEGER,
    status VARCHAR(20) DEFAULT '未开始',
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    start_time DATETIME,
    end_time DATETIME,
    FOREIGN KEY (user_id) REFERENCES users (id),
    FOREIGN KEY (config_id) REFERENCES config_files (id)
);

-- 爬虫结果表 (任务隔离)
CREATE TABLE crawler_results (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    task_id INTEGER NOT NULL,
    content TEXT NOT NULL,      -- JSON结果数据
    is_saved BOOLEAN DEFAULT 0, -- 是否长期保存
    create_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (task_id) REFERENCES crawler_tasks (id)
);

```

数据访问控制

```

# 所有查询都带user_id过滤, 确保数据隔离
class CrawlerTask:
    @classmethod
    def get_by_user(cls, user_id, status=None):
        """只返回该用户的任务"""
        cursor.execute(
            'SELECT * FROM crawler_tasks WHERE user_id = ? ...',
            (user_id,)
        )

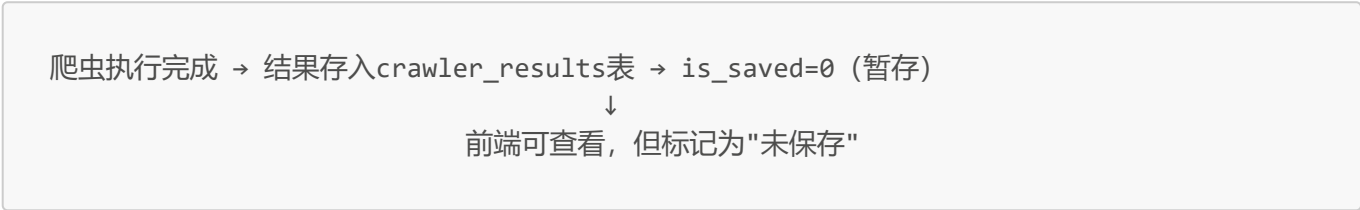
    @classmethod

```

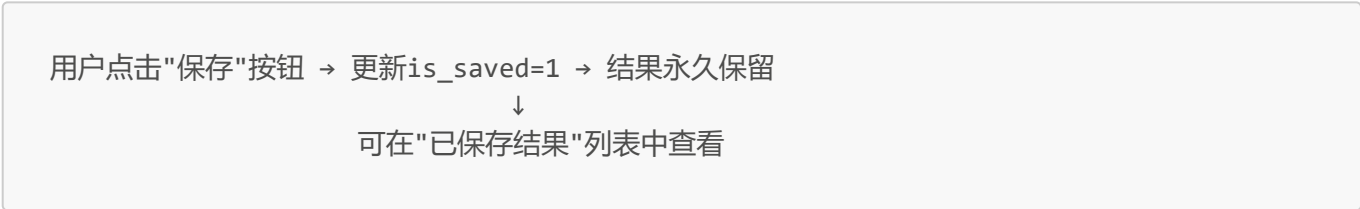
```
def get_by_id(cls, task_id, user_id=None):
    """查询时验证用户权限"""
    task = cls._get_by_id(task_id)
    if user_id and task.user_id != user_id:
        raise PermissionError("无权访问此任务")
    return task
```

存储流程

暂存流程



长期保存流程



数据清理策略

数据类型	清理策略	触发条件
暂存结果	自动清理	7天后自动删除
已保存结果	手动删除	用户主动删除
任务记录	保留	用于审计追踪
配置文件	手动删除	用户主动删除

第六页：可视化服务对接

可视化架构

服务分层

层级	组件	职责
数据获取层	Model Query	从数据库查询原始数据
数据处理层	DataProcessor	数据清洗、格式转换
AI分析层	AIModelCaller	情感分析、关键词提取

层级	组件	职责
图表生成层	DataVisualizer	matplotlib生成图表
展示层	HTML/JS	Base64图片渲染

从数据库到可视化的完整流程

流程图



AI分析双模式

模式对比

特性	本地模型模式	云端API模式
数据隐私	数据不出本地，最安全	数据上传云端
网络依赖	无需外网	需要API连接
响应速度	依赖本地硬件	依赖网络和服务商
模型选择	固定本地模型	灵活选择多种模型
成本	硬件成本	按调用量计费
适用场景	敏感数据、内网环境	通用场景、需要高级模型

配置方式

```
// 本地模型配置
{
  "config_type": "local",
  "model_name": "llama-2-7b",
  "base_url": "http://localhost:8000/v1/chat/completions"
}

// 云端API配置
{
  "config_type": "api",
  "model_name": "gpt-3.5-turbo",
  "base_url": "https://api.openai.com/v1/chat/completions",
  "api_key": "sk-xxxxxxx"
}
```

代码实现

```
class AIModelCaller:
    def __init__(self, config):
        self.config_type = config.get('config_type', 'local')
        self.base_url = config.get('base_url')
        self.api_key = config.get('api_key', '')
        self.model_name = config.get('model_name')

    def analyze(self, data, task_type):
        prompt = self._build_prompt(data, task_type)

        if self.config_type == 'api':
            return self._call_api(prompt)
        else:
            return self._call_local(prompt)
```

```
def _call_api(self, prompt):
    headers = {'Authorization': f'Bearer {self.api_key}'}
    response = requests.post(self.base_url,
                             json={'model': self.model_name, 'messages': [...]},
                             headers=headers)
    return response.json()

def _call_local(self, prompt):
    # 调用本地部署的模型服务
    response = requests.post(self.base_url,
                             json={'prompt': prompt, 'model': self.model_name})
    return response.json()
```

云服务化扩展设计

当前架构的云服务化准备

组件	当前实现	云服务化改造	改动量
可视化服务	本地matplotlib生成	云端图表服务API	小
AI分析	本地/API双模式	统一云端AI服务	无（已支持）
数据存储	SQLite本地文件	云数据库（RDS）	小（仅配置）
文件存储	本地文件系统	对象存储（OSS）	小
前端展示	Base64图片	CDN图片链接	小

可视化服务云化方案

方案：可视化作为独立微服务

当前模式：
前端 → Flask后端 → matplotlib生成 → Base64返回 → 前端展示

云服务模式：
前端 → Flask后端 → 调用可视化服务API → 返回图片URL → 前端展示

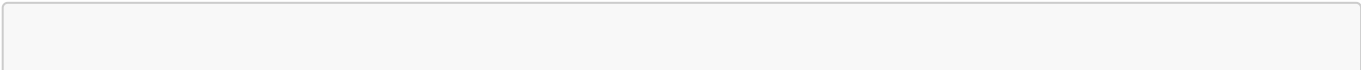
↓

可视化微服务（独立部署）

- 接收数据
- 生成图表
- 上传OSS
- 返回URL

改造点说明

1. 后端调用方式修改（仅1处）



```
# 当前代码
from show import visualize_data
charts = visualize_data(data) # 返回Base64

# 云服务化后
import requests
def visualize_data(data):
    response = requests.post('https://viz-service.cloud.com/api/charts',
                             json={'data': data})
    return response.json()['image_urls'] # 返回OSS URL
```

2. 前端展示方式修改（仅1处）

```
<!-- 当前代码 -->


<!-- 云服务化后 -->

```

3. 其余代码无需改动

- 数据模型层
- 爬虫服务层
- AI分析层
- 用户认证层

云服务化优势

优势	说明
解耦	可视化服务独立演进
扩展	支持更多图表类型（ECharts、D3.js等）
性能	图表生成不占用主服务资源
缓存	相同数据直接返回缓存图片
成本	按需使用，降低服务器压力

第七页：总结与展望

项目亮点总结

维度	亮点
架构	分层清晰，云服务化就绪
爬虫	配置灵活，支持嵌套选择器

维度	亮点
AI	本地/云端双模式，数据安全可控
可视化	6种图表类型，中文完美支持
安全	多用户隔离，bcrypt加密
协作	三人分工明确，接口清晰

未来规划

短期 (1-2月)

- ☐ 完善异常处理和日志系统
- ☐ 增加更多图表类型
- ☐ 优化前端交互体验

中期 (3-6月)

- ☐ 可视化服务云化改造
- ☐ 支持分布式爬虫
- ☐ 接入更多AI模型

长期 (6-12月)

- ☐ 完整SaaS化产品
- ☐ 多租户架构
- ☐ 商业化运营

附录：接口速查表

核心API列表

接口	方法	功能	负责人
/api/register	POST	用户注册	A
/api/login	POST	用户登录	A
/api/config/upload	POST	上传配置	A
/api/task/create	POST	创建任务	A
/api/task//start	POST	启动爬虫	A
/api/ai/analyze	POST	AI分析	B
/api/result//visualize	GET	生成可视化	C

配置文件模板

详见项目目录：

- crawl_demo_config.json - 爬虫配置示例
- time_demo_config.json - 定时配置示例

演示结束，感谢聆听！

Q&A环节